

Blackthorn

Security Review For Morpho

Collaborative Audit Prepared For: **Morpho**

Lead Security Expert(s): **0x73696d616f**

hyh

xiaoming90

Date Audited: **August 19 - August 20, 2026**

Introduction

The `BlueBundlesV1` contract provides opinionated entrypoints to the Morpho Blue lending protocol.

Scope

Repository: `morpho-org/bundles`

Audited Commit: `ae0b59ed3e8c57db3a2ff68a85971debf8b5c5a1`

Final Commit: `068d625de4623a522b25196928fc4b053a953141`

Files:

- `src/blue/BlueBundlesV1.sol`
- `src/blue/interfaces/IBlueBundlesV1.sol`

Final Commit Hash

068d625de4623a522b25196928fc4b053a953141

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	2

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x73696d6166

A handwritten signature in white ink on a black background. The signature consists of a large, stylized capital 'S' that loops around the word 'Simao', which is written in a cursive script.

@hyh

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'hyh'.

@xiaoming90

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'Xiaoming90'.

Issue L-1: Missing output bound in blueBundlesV1Withdraw lets stale fixed penalty consume collapsed withdrawal [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-08-morpho-blue-bundles-update-aug-19th-2026/issues/3>

Vulnerability Detail

blueBundlesV1Withdraw pre-commits the caller to a fixed Public-Allocator penalty P at the start of the execution in Line 227 below

```
File: BlueBundlesV1.sol
212:     function blueBundlesV1Withdraw(
..SNIP..
221:     ) external {
222:         require(block.timestamp <= deadline, DeadlinePassed());
223:         require(referralFeePct < WAD, PctExceeded());
224:
225:         setAuthorizationWithSig(signedAuthorization);
226:
227:         uint256 penaltyAssets = totalPenaltyAssets(reallocations);
..SNIP..
File: BlueBundlesV1.sol
451:     function totalPenaltyAssets(PublicAllocations[] memory reallocations)
↪ internal pure returns (uint256) {
452:         uint256 penaltyAssets;
453:         for (uint256 i; i < reallocations.length; i++) {
454:             penaltyAssets +=
↪ uint256(reallocations[i].assets).mulDivUp(reallocations[i].penalty, WAD);
455:         }
456:         return penaltyAssets;
457:     }
```

However, the gross withdrawal W is computed from the live share price at execution,

which may decline due to bad debt incurred. So if bad debt collapses the share price between quoting and inclusion, the caller irrevocably pays a penalty sized for a withdrawal that no longer exists

Consider the following scenario:

1. Quote: $W_0 = 1,000,000$; liquidity 300,000 $\Rightarrow$ size allocation $R = 703,518$ @ 0.5% so the net inflow $R - P$ covers the 700,000 gap $\Rightarrow P = 3,518$ baked into calldata.
2. Before inclusion: permissionless `liquidate()` writes off 700,000 bad debt (zero repayment). W collapses to exactly 300,000.
3. Bundle executes anyway: pays $P = 3,518$, allocates 703,518 nobody needs, withdraws $W = 300,000$.
4. Caller receives 296,482. The 3,518 is gone, which paid for an allocation that the withdrawal no longer required.

Impact

An honest caller whose withdrawal proceeds collapse due to bad debt between quote and inclusion irrevocably pays the stale penalty P , with no parameter to decline. Loss is bounded by P but can reach 100% of the position's remaining value, and the penalty rate is uncapped.

Code Snippet

https://github.com/sherlock-audit/2026-08-morpho-blue-bundles-update-aug-19th-2026/blob/521233780a2baad13bf32b0674b67a77d6464368/morpho-org__bundles/src/blue/BlueBundlesV1.sol#L212

Tool Used

Manual Review

Recommendation

Consider adding the `minSharePriceE27` to `blueBundlesV1Withdraw`.

```
require(withdrawnAssetsTransient.mulDivDown(1e27, shares) >= minSharePriceE27,  
↪ SlippageExceeded());
```

Walking through the previous scenario with `minSharePriceE27` fix:

1. Quote: same calldata, plus `minSharePriceE27` set near the quoted price.
2. Bad debt lands; realized share price is far below the floor.
3. Bound trips $\boxtimes$ revert. Cost: gas only. P is never paid (whole bundle unwinds atomically).
4. Re-quote: $W = 300,000 \boxtimes$ liquidity 300,000 $\Rightarrow$ no reallocation needed, $P \boxtimes = 0$. Caller receives the full 300,000 instead of 296,482

Discussion

bhargavbh

good catch and we will document it. We want PA executions to be static (i.e. execute all the reallocations) rather than dynamic (i.e. only execute on-demand the reallocations necessary). With the static PA design, we can't achieve the two together:

1. no shareprice slippage check
2. slippage check on penalty

We would rather have 1 than 2.

Issue L-2: Collateral cannot be supplied through BlueBundlesV1 without borrowing [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-08-morpho-blue-bundles-update-aug-19th-2026/issues/4>

Summary

BlueBundlesV1::blueBundlesV1SupplyCollateralAndBorrow() checks if (`collateralAssets > 0`) but doesn't do the same for borrowing, so a call with `borrowAssets == 0` always reverts.

Vulnerability Detail

BlueBundlesV1::blueBundlesV1SupplyCollateralAndBorrow() skips the supply when no collateral is passed, which is what makes a pure borrow work:

```
if (collateralAssets > 0) {
    TokenLib.forceApproveMax(marketParams.collateralToken, BLUE);
    IMorpho(BLUE).supplyCollateral(marketParams, collateralAssets, msg.sender, "");
}
```

However, there is no matching check for pure collateralization.

BlueBundlesV1::executeBorrow() calls Morpho Blue's borrow with assets set to `borrowAssets` and shares set to 0. Morpho Blue requires exactly one of the two to be zero, so `borrowAssets == 0` reverts with inconsistent input.

Impact

Adding collateral without borrowing is impossible through the bundler.

Code Snippet

BlueBundlesV1::blueBundlesV1SupplyCollateralAndBorrow() and BlueBundlesV1::executeBorrow().

Tool Used

Manual Review

Recommendation

Skip borrowing if the amount is null.

Discussion

bhargavbh

resolved in <https://github.com/morpho-org/bundles/pull/125>

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.